Gebze Technical University

Department of Computer Engineering

CSE344 - System Programming

Homework #5 Report

Priority-Based Cargo Delivery Simulation

Student: Halit Batuhan ASLAN

Student ID: 220104004003

Date: May 2026

Contents

1 System Design

The program implements a fixed-size courier pool. All valid orders are read from the input file at startup, inserted into a shared priority queue, and only then courier threads are created. No new courier thread is created while the shift is running.

Each courier repeatedly takes one order from the queue, delivers it by sleeping for the requested simulation time, updates statistics, and then returns to the queue. When the queue is empty and no active delivery remains, all couriers leave the loop and the main thread writes the final statistics file.

In the updated assignment, one simulation unit is 500 milliseconds. The implementation uses one shared constant for this conversion, so console durations, total delivery time, courier totals, and sleeping time are calculated with the same value.

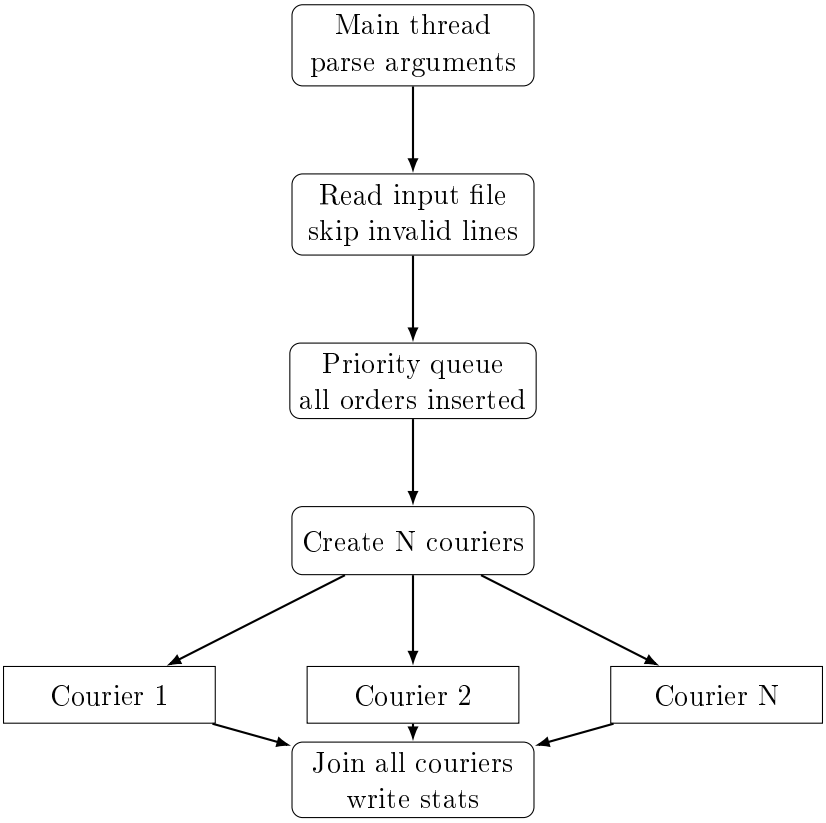

Figure 1: General program flow

1.1 Courier Lifecycle

A courier starts in the depot, locks the queue mutex, and tries to pop the best available order. If it cannot work yet, it waits on the condition variable. After an order is taken, the courier releases the queue mutex before sleeping, so other couriers can continue to take work. At the end of a delivery, the courier updates atomic totals and its own local statistics.

2 Priority Queue Design

The priority queue is implemented as a binary min-heap. The comparison function first compares priority level and then order id. Since EXPRESS is represented with the smallest value, it naturally comes before STANDARD and ECONOMY. If two orders have the same priority, the lower id is placed earlier.

Priority	Internal value	Meaning
EXPRESS	1	Highest priority
STANDARD	2	Normal priority
ECONOMY	3	Lowest priority

Table 1: Priority mapping

The courier logs `DELIVERY_START` immediately after popping the order while still holding the queue mutex. This keeps the visible start order consistent with the scheduling decision.

3 Synchronization Strategy

Primitive	Purpose
`queue_mutex`	Protects the priority queue, active delivery count, and shutdown flag.
`queue_cond`	Blocks idle couriers and wakes them when the shift ends or shutdown starts.
`log_mutex`	Protects every stdout line from interleaving.
`_Atomic` counters	Tracks completed orders, cancelled orders, and total delivery time without a mutex.

Table 2: Synchronization primitives

There is no busy waiting. Couriers use `pthread_cond_wait()` when they need to wait. The main thread uses a timed condition wait only to notice the SIGINT flag without doing unsafe work inside the signal handler.

4 SIGINT Handling

The signal handler only sets a `sig_atomic_t` flag. It does not print, lock a mutex, allocate memory, or cancel threads. The main thread observes this flag in normal control flow and performs the shutdown sequence.

1. SIGINT arrives and the handler sets the flag.
2. The main thread leaves its wait loop.
3. The program enters shutdown mode under `queue_mutex`.
4. Pending orders are removed from the queue and logged as cancelled.
5. Waiting couriers are awakened with `pthread_cond_broadcast()`.
6. Couriers already delivering finish their current order.
7. The main thread joins every courier and writes the final summary.

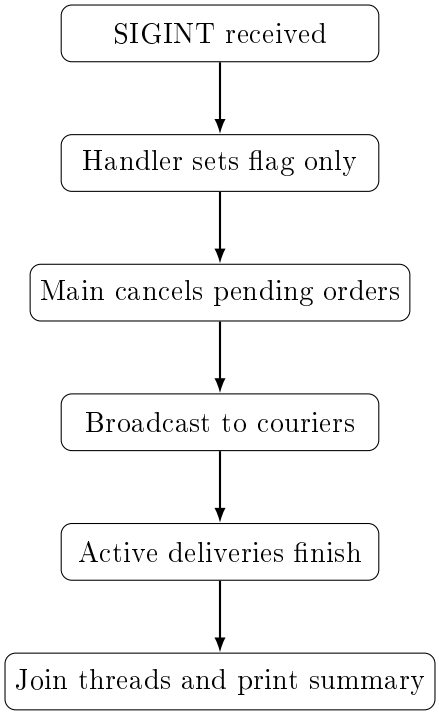

Figure 2: SIGINT shutdown sequence

5 Test Scenarios

The following commands were used for the required scenarios. Each screenshot should show the complete terminal area from the command prompt to program exit.

5.1 Scenario 1: Mixed 10 Orders

```
./cargoGTU -n 3 -i 10orders_mix.txt -s stats.txt
```

This test checks that all 10 orders are completed and that `stats.txt` is written correctly. The updated homework removed the older requirement about all EXPRESS start lines appearing before the other priority classes in this scenario.

```
bash: cargoGTU: command not found
halit@debian:~/MyHW/Homework5$ ./cargoGTU -n 3 -i 10orders_mix.txt -s stats.txt
[CARGOGTU] SHIFT_START couriers=3 orders=10
[CARGOGTU] ORDER_QUEUED id=1 recipient=Ahmet_Yilmaz priority=EXPRESS duration=8
[CARGOGTU] ORDER_QUEUED id=2 recipient=Ayse_Kaya priority=STANDARD duration=12
[CARGOGTU] ORDER_QUEUED id=3 recipient=Mehmet_Demir priority=ECONOMY duration=20
[CARGOGTU] ORDER_QUEUED id=4 recipient=Fatma_Sahin priority=EXPRESS duration=6
[CARGOGTU] ORDER_QUEUED id=5 recipient=Ali_Celik priority=STANDARD duration=9
[CARGOGTU] ORDER_QUEUED id=6 recipient=Zeynep_Arslan priority=ECONOMY duration=15
[CARGOGTU] ORDER_QUEUED id=7 recipient=Mustafa_Koc priority=EXPRESS duration=7
[CARGOGTU] ORDER_QUEUED id=8 recipient=Elif_Demir priority=STANDARD duration=11
[CARGOGTU] ORDER_QUEUED id=9 recipient=Hasan_Ozturk priority=ECONOMY duration=18
[CARGOGTU] ORDER_QUEUED id=10 recipient=Merve_Yildiz priority=STANDARD duration=10
[COURIER-3] DELIVERY_START id=1 recipient=Ahmet_Yilmaz priority=EXPRESS
[COURIER-2] DELIVERY_START id=4 recipient=Fatma_Sahin priority=EXPRESS
[COURIER-1] DELIVERY_START id=7 recipient=Mustafa_Koc priority=EXPRESS
[COURIER-2] DELIVERY_COMPLETE id=4 recipient=Fatma_Sahin duration=3000ms
[COURIER-2] DELIVERY_START id=2 recipient=Ayse_Kaya priority=STANDARD
[COURIER-1] DELIVERY_COMPLETE id=7 recipient=Mustafa_Koc duration=3500ms
[COURIER-1] DELIVERY_START id=5 recipient=Ali_Celik priority=STANDARD
[COURIER-3] DELIVERY_COMPLETE id=1 recipient=Ahmet_Yilmaz duration=4000ms
[COURIER-3] DELIVERY_START id=8 recipient=Elif_Demir priority=STANDARD
[COURIER-1] DELIVERY_COMPLETE id=5 recipient=Ali_Celik duration=4500ms
[COURIER-1] DELIVERY_START id=10 recipient=Merve_Yildiz priority=STANDARD
[COURIER-2] DELIVERY_COMPLETE id=2 recipient=Ayse_Kaya duration=6000ms
[COURIER-2] DELIVERY_START id=3 recipient=Mehmet_Demir priority=ECONOMY
[COURIER-3] DELIVERY_COMPLETE id=8 recipient=Elif_Demir duration=5500ms
[COURIER-3] DELIVERY_START id=6 recipient=Zeynep_Arslan priority=ECONOMY
[COURIER-1] DELIVERY_COMPLETE id=10 recipient=Merve_Yildiz duration=5000ms
[COURIER-1] DELIVERY_START id=9 recipient=Hasan_Ozturk priority=ECONOMY
[COURIER-3] DELIVERY_COMPLETE id=6 recipient=Zeynep_Arslan duration=7500ms
[COURIER-3] WAITING
[COURIER-2] DELIVERY_COMPLETE id=3 recipient=Mehmet_Demir duration=10000ms
[COURIER-2] WAITING
[COURIER-1] DELIVERY_COMPLETE id=9 recipient=Hasan_Ozturk duration=9000ms
[COURIER-1] SHIFT_OVER
[COURIER-3] SHIFT_OVER
[COURIER-2] SHIFT_OVER
[CARGOGTU] SHIFT_END completed=10 cancelled=0 total_time=58000ms
[CARGOGTU] SHUTDOWN_COMPLETE
halit@debian:~/MyHW/Homework5$
```

Figure 3: Scenario 1 console output

5.2 Scenario 2: Economy Only

```
./cargoGTU -n 10 -i 10orders_economy.txt -s stats.txt
```

This test checks the tie rule. Since all orders have the same priority, DELIVERY_START lines must follow increasing id order.

```
halit@debian:~/MyHW/Homework5$ ./cargoGTU -n 10 -i 10orders_economy.txt -s stats.txt
[CARGOGTU] SHIFT_START couriers=10 orders=10
[CARGOGTU] ORDER_QUEUED id=1 recipient=Ahmet_Yilmaz priority=ECONOMY duration=8
[CARGOGTU] ORDER_QUEUED id=2 recipient=Ayse_Kaya priority=ECONOMY duration=10
[CARGOGTU] ORDER_QUEUED id=3 recipient=Mehmet_Demir priority=ECONOMY duration=7
[CARGOGTU] ORDER_QUEUED id=4 recipient=Fatma_Sahin priority=ECONOMY duration=12
[CARGOGTU] ORDER_QUEUED id=5 recipient=Ali_Celik priority=ECONOMY duration=9
[CARGOGTU] ORDER_QUEUED id=6 recipient=Zeynep_Arslan priority=ECONOMY duration=11
[CARGOGTU] ORDER_QUEUED id=7 recipient=Mustafa_Koc priority=ECONOMY duration=8
[CARGOGTU] ORDER_QUEUED id=8 recipient=Elif_Demir priority=ECONOMY duration=6
[CARGOGTU] ORDER_QUEUED id=9 recipient=Hasan_Ozturk priority=ECONOMY duration=10
[CARGOGTU] ORDER_QUEUED id=10 recipient=Merve_Yildiz priority=ECONOMY duration=9
[COURIER-7] DELIVERY_START id=1 recipient=Ahmet_Yilmaz priority=ECONOMY
[COURIER-8] DELIVERY_START id=2 recipient=Ayse_Kaya priority=ECONOMY
[COURIER-9] DELIVERY_START id=3 recipient=Mehmet_Demir priority=ECONOMY
[COURIER-10] DELIVERY_START id=4 recipient=Fatma_Sahin priority=ECONOMY
[COURIER-6] DELIVERY_START id=5 recipient=Ali_Celik priority=ECONOMY
[COURIER-5] DELIVERY_START id=6 recipient=Zeynep_Arslan priority=ECONOMY
[COURIER-4] DELIVERY_START id=7 recipient=Mustafa_Koc priority=ECONOMY
[COURIER-3] DELIVERY_START id=8 recipient=Elif_Demir priority=ECONOMY
[COURIER-2] DELIVERY_START id=9 recipient=Hasan_Ozturk priority=ECONOMY
[COURIER-1] DELIVERY_START id=10 recipient=Merve_Yildiz priority=ECONOMY
[COURIER-3] DELIVERY_COMPLETE id=8 recipient=Elif_Demir duration=3000ms
[COURIER-3] WAITING
[COURIER-9] DELIVERY_COMPLETE id=3 recipient=Mehmet_Demir duration=3500ms
[COURIER-9] WAITING
[COURIER-7] DELIVERY_COMPLETE id=1 recipient=Ahmet_Yilmaz duration=4000ms
[COURIER-7] WAITING
[COURIER-4] DELIVERY_COMPLETE id=7 recipient=Mustafa_Koc duration=4000ms
[COURIER-4] WAITING
[COURIER-6] DELIVERY_COMPLETE id=5 recipient=Ali_Celik duration=4500ms
[COURIER-6] WAITING
[COURIER-1] DELIVERY_COMPLETE id=10 recipient=Merve_Yildiz duration=4500ms
[COURIER-1] WAITING
[COURIER-8] DELIVERY_COMPLETE id=2 recipient=Ayse_Kaya duration=5000ms
[COURIER-8] WAITING
[COURIER-2] DELIVERY_COMPLETE id=9 recipient=Hasan_Ozturk duration=5000ms
[COURIER-2] WAITING
[COURIER-5] DELIVERY_COMPLETE id=6 recipient=Zeynep_Arslan duration=5500ms
[COURIER-5] WAITING
[COURIER-10] DELIVERY_COMPLETE id=4 recipient=Fatma_Sahin duration=6000ms
[COURIER-10] SHIFT_OVER
[COURIER-3] SHIFT_OVER
[COURIER-9] SHIFT_OVER
[COURIER-7] SHIFT_OVER
[COURIER-4] SHIFT_OVER
[COURIER-6] SHIFT_OVER
[COURIER-1] SHIFT_OVER
[COURIER-8] SHIFT_OVER
[COURIER-2] SHIFT_OVER
[COURIER-5] SHIFT_OVER
[CARGOGTU] SHIFT_END completed=10 cancelled=0 total_time=45000ms
[CARGOGTU] SHUTDOWN_COMPLETE
halit@debian:~/MyHW/Homework5$
```

Figure 4: Scenario 2 console output

5.3 Scenario 3: SIGINT

```
./cargoGTU -n 2 -i 20orders_mix.txt -s stats.txt
# send Ctrl+C after about 2 seconds
```

This test checks that active deliveries finish, pending orders are cancelled, and completed plus cancelled equals the total number of valid orders.

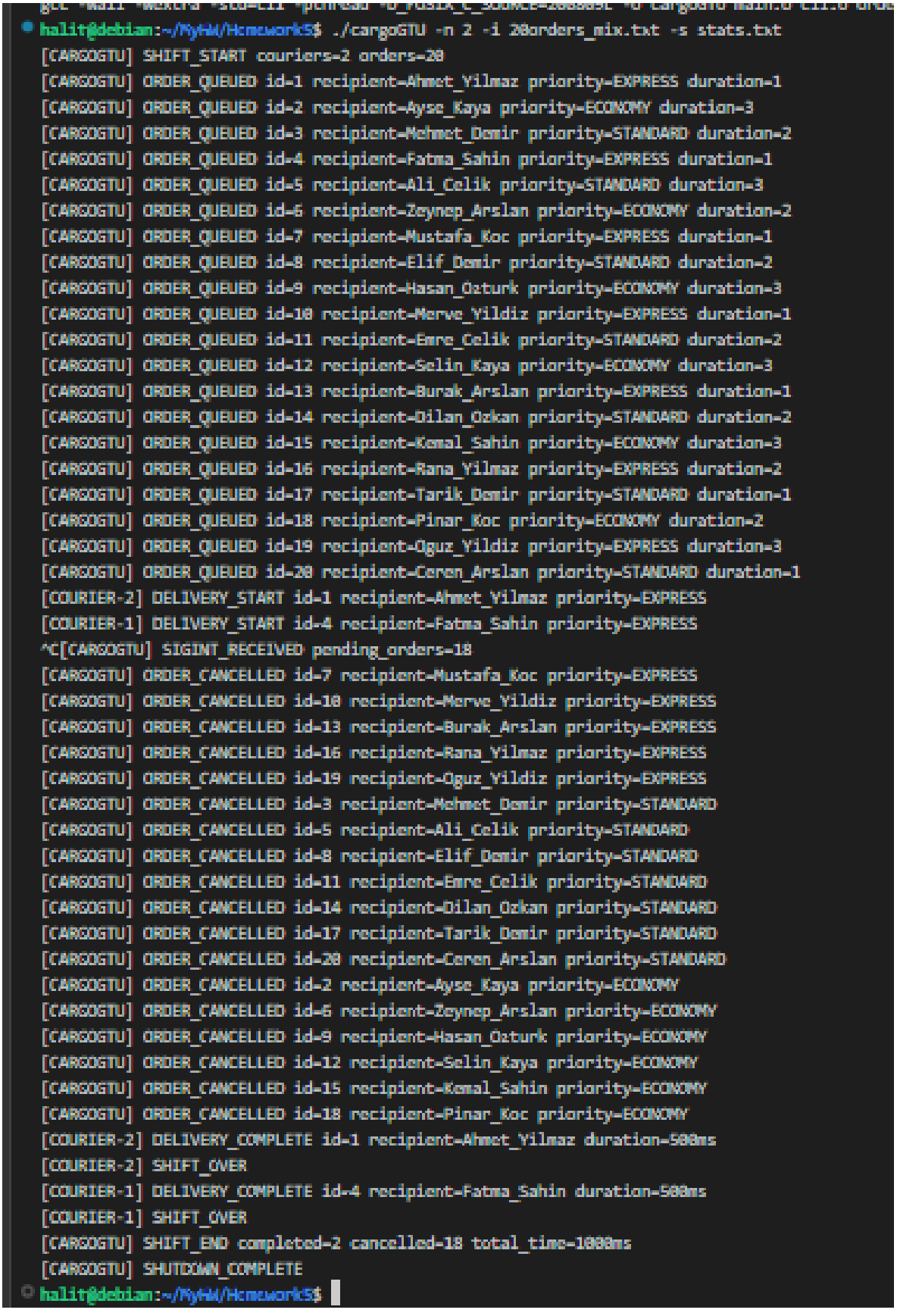

Figure 5: Scenario 3 console output

5.4 Scenario 4: Valgrind

```
valgrind --leak-check=full ./cargoGTU -n 3 -i 10orders_mix.txt -s stats.
    txt
```

This test checks memory cleanup. The expected result is zero errors and zero leaks.

```
halit@debian:~/MyHW/Homework5$ valgrind --leak-check=full ./cargoGTU -n 3 -i 10orders_mix.txt -s stats.txt
==1379== Memcheck, a memory error detector
==1379== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==1379== Using Valgrind-3.16.1 and LibVEX; rerun with -h for copyright info
==1379== Command: ./cargoGTU -n 3 -i 10orders_mix.txt -s stats.txt
==1379==
[CARGOGTU] SHIFT_START couriers=3 orders=10
[CARGOGTU] ORDER_QUEUED id=1 recipient=Ahmet_Yilmaz priority=EXPRESS duration=8
[CARGOGTU] ORDER_QUEUED id=2 recipient=Ayse_Kaya priority=STANDARD duration=12
[CARGOGTU] ORDER_QUEUED id=3 recipient=Mehmet_Demir priority=ECONOMY duration=20
[CARGOGTU] ORDER_QUEUED id=4 recipient=Fatma_Sahin priority=EXPRESS duration=6
[CARGOGTU] ORDER_QUEUED id=5 recipient=Ali_Celik priority=STANDARD duration=9
[CARGOGTU] ORDER_QUEUED id=6 recipient=Zeynep_Arslan priority=ECONOMY duration=15
[CARGOGTU] ORDER_QUEUED id=7 recipient=Mustafa_Koc priority=EXPRESS duration=7
[CARGOGTU] ORDER_QUEUED id=8 recipient=Elif_Demir priority=STANDARD duration=11
[CARGOGTU] ORDER_QUEUED id=9 recipient=Hasan_Ozturk priority=ECONOMY duration=18
[CARGOGTU] ORDER_QUEUED id=10 recipient=Merve_Yildiz priority=STANDARD duration=10
[COURIER-1] DELIVERY_START id=1 recipient=Ahmet_Yilmaz priority=EXPRESS
[COURIER-2] DELIVERY_START id=4 recipient=Fatma_Sahin priority=EXPRESS
[COURIER-3] DELIVERY_START id=7 recipient=Mustafa_Koc priority=EXPRESS
[COURIER-2] DELIVERY_COMPLETE id=4 recipient=Fatma_Sahin duration=3000ms
[COURIER-2] DELIVERY_START id=2 recipient=Ayse_Kaya priority=STANDARD
[COURIER-3] DELIVERY_COMPLETE id=7 recipient=Mustafa_Koc duration=3500ms
[COURIER-3] DELIVERY_START id=5 recipient=Ali_Celik priority=STANDARD
[COURIER-1] DELIVERY_COMPLETE id=1 recipient=Ahmet_Yilmaz duration=4000ms
[COURIER-1] DELIVERY_START id=8 recipient=Elif_Demir priority=STANDARD
[COURIER-3] DELIVERY_COMPLETE id=5 recipient=Ali_Celik duration=4500ms
[COURIER-3] DELIVERY_START id=10 recipient=Merve_Yildiz priority=STANDARD
[COURIER-2] DELIVERY_COMPLETE id=2 recipient=Ayse_Kaya duration=6000ms
[COURIER-2] DELIVERY_START id=3 recipient=Mehmet_Demir priority=ECONOMY
[COURIER-1] DELIVERY_COMPLETE id=8 recipient=Elif_Demir duration=5500ms
[COURIER-1] DELIVERY_START id=6 recipient=Zeynep_Arslan priority=ECONOMY
[COURIER-3] DELIVERY_COMPLETE id=10 recipient=Merve_Yildiz duration=5000ms
[COURIER-3] DELIVERY_START id=9 recipient=Hasan_Ozturk priority=ECONOMY
[COURIER-1] DELIVERY_COMPLETE id=6 recipient=Zeynep_Arslan duration=7500ms
[COURIER-1] WAITING
[COURIER-2] DELIVERY_COMPLETE id=3 recipient=Mehmet_Demir duration=10000ms
[COURIER-2] WAITING
[COURIER-3] DELIVERY_COMPLETE id=9 recipient=Hasan_Ozturk duration=9000ms
[COURIER-3] SHIFT_OVER
[COURIER-1] SHIFT_OVER
[COURIER-2] SHIFT_OVER
[CARGOGTU] SHIFT_END completed=10 cancelled=0 total_time=58000ms
[CARGOGTU] SHUTDOWN_COMPLETE
==1379==
==1379== HEAP SUMMARY:
==1379==     in use at exit: 0 bytes in 0 blocks
==1379==   total heap usage: 13 allocs, 13 frees, 12,632 bytes allocated
==1379==
==1379== All heap blocks were freed -- no leaks are possible
==1379==
==1379== For lists of detected and suppressed errors, rerun with: -s
==1379== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
halit@debian:~/MyHW/Homework5$
```

Figure 6: Scenario 4 Valgrind output

5.5 Scenario 5: High Concurrency

```
./cargoGTU -n 10 -i 20orders_mix.txt -s stats.txt
```

This test is run multiple times. The completed count must stay the same, no order id should appear twice in DELIVERY_START, and priority order must still be preserved.

```
halit@debian:~/MyHW/HomeworkS$ ./cargoGTU -n 10 -i 20orders_mix.txt -s stats.txt
[CARGOGTU] SHIFT_START couriers=10 orders=20
[CARGOGTU] ORDER_QUEUED id=1 recipient=Ahmet_Yilmaz priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=2 recipient=Ayse_Kaya priority=ECONOMY duration=3
[CARGOGTU] ORDER_QUEUED id=3 recipient=Mehmet_Demir priority=STANDARD duration=2
[CARGOGTU] ORDER_QUEUED id=4 recipient=Fatma_Sahin priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=5 recipient=Ali_Celik priority=STANDARD duration=3
[CARGOGTU] ORDER_QUEUED id=6 recipient=Zeynep_Arslan priority=ECONOMY duration=2
[CARGOGTU] ORDER_QUEUED id=7 recipient=Mustafa_Koc priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=8 recipient=Elif_Demir priority=STANDARD duration=2
[CARGOGTU] ORDER_QUEUED id=9 recipient=Hasan_Ozturk priority=ECONOMY duration=3
[CARGOGTU] ORDER_QUEUED id=10 recipient=Merve_Yildiz priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=11 recipient=Emre_Celik priority=STANDARD duration=2
[CARGOGTU] ORDER_QUEUED id=12 recipient=Selin_Kaya priority=ECONOMY duration=3
[CARGOGTU] ORDER_QUEUED id=13 recipient=Burak_Arslan priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=14 recipient=Dilan_Ozkan priority=STANDARD duration=2
[CARGOGTU] ORDER_QUEUED id=15 recipient=Kemal_Sahin priority=ECONOMY duration=3
[CARGOGTU] ORDER_QUEUED id=16 recipient=Rana_Yilmaz priority=EXPRESS duration=2
[CARGOGTU] ORDER_QUEUED id=17 recipient=Tarik_Demir priority=STANDARD duration=1
[CARGOGTU] ORDER_QUEUED id=18 recipient=Pinar_Koc priority=ECONOMY duration=2
[CARGOGTU] ORDER_QUEUED id=19 recipient=Oguz_Yildiz priority=EXPRESS duration=3
[CARGOGTU] ORDER_QUEUED id=20 recipient=Ceren_Arslan priority=STANDARD duration=1
[COURIER-7] DELIVERY_START id=1 recipient=Ahmet_Yilmaz priority=EXPRESS
[COURIER-8] DELIVERY_START id=4 recipient=Fatma_Sahin priority=EXPRESS
[COURIER-9] DELIVERY_START id=7 recipient=Mustafa_Koc priority=EXPRESS
[COURIER-10] DELIVERY_START id=10 recipient=Merve_Yildiz priority=EXPRESS
[COURIER-6] DELIVERY_START id=13 recipient=Burak_Arslan priority=EXPRESS
[COURIER-5] DELIVERY_START id=16 recipient=Rana_Yilmaz priority=EXPRESS
[COURIER-4] DELIVERY_START id=19 recipient=Oguz_Yildiz priority=EXPRESS
[COURIER-3] DELIVERY_START id=3 recipient=Mehmet_Demir priority=STANDARD
[COURIER-2] DELIVERY_START id=5 recipient=Ali_Celik priority=STANDARD
[COURIER-1] DELIVERY_START id=8 recipient=Elif_Demir priority=STANDARD
[COURIER-7] DELIVERY_COMPLETE id=1 recipient=Ahmet_Yilmaz duration=500ms
[COURIER-7] DELIVERY_START id=11 recipient=Emre_Celik priority=STANDARD
[COURIER-8] DELIVERY_COMPLETE id=4 recipient=Fatma_Sahin duration=500ms
[COURIER-8] DELIVERY_START id=14 recipient=Dilan_Ozkan priority=STANDARD
[COURIER-9] DELIVERY_COMPLETE id=7 recipient=Mustafa_Koc duration=500ms
[COURIER-9] DELIVERY_START id=17 recipient=Tarik_Demir priority=STANDARD
[COURIER-10] DELIVERY_COMPLETE id=10 recipient=Merve_Yildiz duration=500ms
[COURIER-10] DELIVERY_START id=20 recipient=Ceren_Arslan priority=STANDARD
[COURIER-6] DELIVERY_COMPLETE id=13 recipient=Burak_Arslan duration=500ms
[COURIER-6] DELIVERY_START id=2 recipient=Ayse_Kaya priority=ECONOMY
[COURIER-5] DELIVERY_COMPLETE id=16 recipient=Rana_Yilmaz duration=1000ms
[COURIER-5] DELIVERY_START id=6 recipient=Zeynep_Arslan priority=ECONOMY
[COURIER-3] DELIVERY_COMPLETE id=3 recipient=Mehmet_Demir duration=1000ms
[COURIER-3] DELIVERY_START id=9 recipient=Hasan_Ozturk priority=ECONOMY
[COURIER-9] DELIVERY_COMPLETE id=17 recipient=Tarik_Demir duration=500ms
[COURIER-9] DELIVERY_START id=12 recipient=Selin_Kaya priority=ECONOMY
[COURIER-10] DELIVERY_COMPLETE id=20 recipient=Ceren_Arslan duration=500ms
[COURIER-10] DELIVERY_START id=15 recipient=Kemal_Sahin priority=ECONOMY
[COURIER-1] DELIVERY_COMPLETE id=8 recipient=Elif_Demir duration=1000ms
[COURIER-1] DELIVERY_START id=18 recipient=Pinar_Koc priority=ECONOMY
[COURIER-7] DELIVERY_COMPLETE id=11 recipient=Emre_Celik duration=1000ms
[COURIER-7] WAITING
[COURIER-8] DELIVERY_COMPLETE id=14 recipient=Dilan_Ozkan duration=1000ms
[COURIER-8] WAITING
[COURIER-4] DELIVERY_COMPLETE id=19 recipient=Oguz_Yildiz duration=1500ms
[COURIER-4] WAITING
[COURIER-2] DELIVERY_COMPLETE id=5 recipient=Ali_Celik duration=1500ms
[COURIER-2] WAITING
[COURIER-6] DELIVERY_COMPLETE id=2 recipient=Ayse_Kaya duration=1500ms
[COURIER-6] WAITING
[COURIER-5] DELIVERY_COMPLETE id=6 recipient=Zeynep_Arslan duration=1000ms
[COURIER-5] WAITING
[COURIER-1] DELIVERY_COMPLETE id=18 recipient=Pinar_Koc duration=1000ms
[COURIER-1] WAITING
[COURIER-3] DELIVERY_COMPLETE id=9 recipient=Hasan_Ozturk duration=1500ms
```

6 Challenges and Solutions

6.1 Keeping the visible order correct

With several couriers, it is possible for one courier to pop an order and another courier to print first. To avoid that, the program prints `DELIVERY_START` while the queue mutex is still held. The delivery itself starts after the mutex is released, so concurrency is not blocked for the delivery duration.

6.2 Handling SIGINT safely

Calling complex functions from a signal handler is unsafe in a multi-threaded program. The handler in this implementation only sets a flag. The main thread later cancels pending orders, wakes couriers, joins them, and writes the final files.

6.3 Avoiding duplicate work

Only one courier can pop from the priority queue at a time because the queue is protected by `queue_mutex`. Once an order is removed, it no longer exists in the queue, so another courier cannot take the same order.

7 Conclusion

The implementation uses a fixed thread pool, a protected priority queue, condition variables for blocking, C11 atomic counters for shared statistics, and a clean shutdown path for SIGINT. The code is separated into small modules so parsing, scheduling, synchronization, logging, and statistics writing are easier to follow.